

# Chapter 8

*Swagger / OpenAPI*

---

Giving FoodBridge an interactive, self-documenting API surface — and wiring JWT authentication straight into the browser.

---

## 1. What is Swagger?

---

Swagger (OpenAPI) is a tool used to generate interactive API documentation for REST APIs. It provides a web interface where developers can:

- View all available APIs
- Read API documentation
- Test APIs directly from the browser
- Send authenticated requests using JWT

## 2. Why Do We Use Swagger?

---

- Automatically generates API documentation
- Eliminates the need to remember all endpoints
- Allows API testing without Postman
- Shows request and response models
- Makes backend projects easier to understand
- Helps frontend and backend developers collaborate

## 3-4. Dependency & URL

---

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.13</version>
</dependency>
```

This integrates Swagger UI into a Spring Boot application and generates OpenAPI documentation automatically. After running the application, Swagger UI is available at:

```
http://localhost:8080/swagger-ui/index.html
```

## 5-6. Allowing Swagger Through Spring Security

Initially, Swagger returned **403 Forbidden**. Spring Security was protecting Swagger's endpoints — and since Swagger is itself a client that makes HTTP requests, unpermitted URLs were blocked.

```
.requestMatchers(  
    "/swagger-ui/**",  
    "/v3/api-docs/**"  
) .permitAll()
```

| PATH            | PURPOSE                                                    |
|-----------------|------------------------------------------------------------|
| /swagger-ui/**  | Serves the Swagger UI webpage                              |
| /v3/api-docs/** | Provides the OpenAPI JSON documentation used by Swagger UI |

## 7-9. SwaggerConfig, @Bean & OpenAPI

```
@Configuration  
public class SwaggerConfig {  
  
    @Bean  
    public OpenAPI openAPI(){  
  
    }  
}
```

@Bean creates an OpenAPI object and registers it as a Spring Bean. OpenAPI is the main object representing the API documentation, containing:

|                     |                        |            |         |         |
|---------------------|------------------------|------------|---------|---------|
| Project Information | Security Configuration | Components | Servers | Schemas |
|---------------------|------------------------|------------|---------|---------|

## 10. The Info Object

The `Info` object stores information about the project:

```
@Bean
public OpenAPI openAPI() {

    return new OpenAPI()
        .info(
            new Info()
                .title("FoodBridge API")
                .description("Backend REST API for connecting food donors, NGOs and volunteers.")
                .version("1.0")
        );
}
```

| METHOD                      | PURPOSE                      |
|-----------------------------|------------------------------|
| <code>.title()</code>       | Sets the project title       |
| <code>.description()</code> | Sets the project description |
| <code>.version()</code>     | Sets the API version         |

## 11-12. Components & SecurityScheme

```
.components(
    new Components()
)
```

Components act as a container for reusable OpenAPI objects — Security Schemes, Schemas, Headers, and Responses.

```
.addSecuritySchemes(
    "Bearer Authentication",
    new SecurityScheme()
)
```

This defines the authentication mechanism used by the application.

## 13-15. Configuring the Bearer Scheme

| CONFIGURATION                                | PURPOSE                                                             |
|----------------------------------------------|---------------------------------------------------------------------|
| <code>.type(SecurityScheme.Type.HTTP)</code> | Specifies that authentication uses the HTTP protocol                |
| <code>.scheme("bearer")</code>               | Tells Swagger that authentication uses Bearer Tokens                |
| <code>.bearerFormat("JWT")</code>            | Indicates the Bearer Token is a JSON Web Token (documentation only) |

With `scheme("bearer")` configured, Swagger automatically sends:

```
Authorization: Bearer <JWT>
```

## 16. SecurityRequirement

```
.addSecurityItem(  
    new SecurityRequirement()  
        .addList("Bearer Authentication")  
)
```

This applies the previously defined security scheme to the APIs.

| WITHOUT SECURITYREQUIREMENT                 | WITH SECURITYREQUIREMENT                                     |
|---------------------------------------------|--------------------------------------------------------------|
| Swagger knows about JWT but does not use it | Swagger automatically includes the JWT in protected requests |

## 17. Complete Swagger Configuration

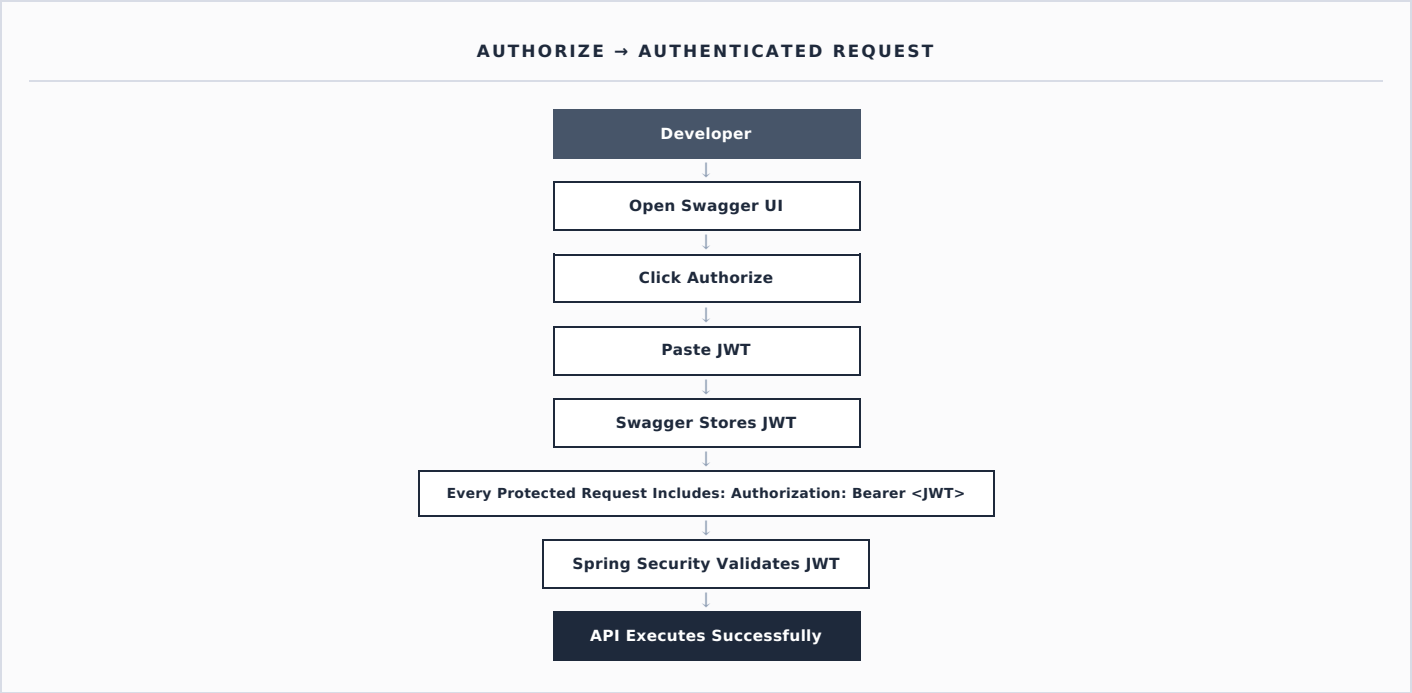
---

```
@Configuration
public class SwaggerConfig {

    @Bean
    public OpenAPI openAPI(){

        return new OpenAPI()
            .info(
                new Info()
                    .title("FoodBridge API")
                    .description("Backend REST API for connecting food donors, NGOs and volunteers.")
                    .version("1.0")
            )
            .components(
                new Components()
                    .addSecuritySchemes(
                        "Bearer Authentication",
                        new SecurityScheme()
                            .type(SecurityScheme.Type.HTTP)
                            .scheme("bearer")
                            .bearerFormat("JWT")
                    )
            )
            .addSecurityItem(
                new SecurityRequirement()
                    .addList("Bearer Authentication")
            );
    }
}
```

## 18. Swagger Authentication Flow



## 19. SecurityScheme vs SecurityRequirement

| SECURITYSCHEME                     | SECURITYREQUIREMENT                                        |
|------------------------------------|------------------------------------------------------------|
| Defines HOW authentication works   | Defines WHERE authentication should be used                |
| Example: Bearer JWT Authentication | Example: Apply Bearer Authentication to all protected APIs |

## 20. Final Flow

### BROWSER TO CONTROLLER AND BACK





## Interview Questions

---

### Q1. What is Swagger?

Swagger is an API documentation tool that generates an interactive web interface for REST APIs. It allows developers to view, understand, and test APIs directly from the browser.

### Q2. Why do we use Swagger?

It automatically documents REST APIs, makes API testing easier, eliminates the need to remember endpoints, and improves collaboration between frontend and backend developers.

### Q3. What is OpenAPI?

OpenAPI is the specification used to describe REST APIs. Swagger is one of the tools that implements the OpenAPI specification.

### Q4. What is the purpose of the OpenAPI Bean?

The OpenAPI Bean stores the project's API documentation, including title, description, version, security configuration, and reusable components.

### Q5. What is SecurityScheme?

SecurityScheme defines how authentication works in the API. In our project, it specifies that authentication uses HTTP Bearer Tokens with JWT.

### Q6. What is SecurityRequirement?

SecurityRequirement applies the configured SecurityScheme to the API endpoints, allowing Swagger to automatically send the JWT with authenticated requests.

### Q7. Why did Swagger return 403 initially?

Spring Security was protecting the Swagger endpoints. We solved this by permitting `/swagger-ui/**` and `/v3/api-docs/**` in SecurityConfig.

### Q8. What happens after clicking the Authorize button?

Swagger stores the JWT and automatically sends it in the Authorization header as `Authorization: Bearer <JWT>` with every protected API request.

## Key Points to Remember

---

- Swagger is an API documentation tool
- OpenAPI is the API specification
- Swagger UI provides an interactive interface for testing APIs
- OpenAPI Bean stores project metadata and configuration
- Info object stores title, description, and version
- Components store reusable OpenAPI objects
- SecurityScheme defines the authentication mechanism
- SecurityRequirement applies authentication to APIs
- Swagger sends JWT using the Authorization header
- Swagger endpoints must be permitted in Spring Security

### CHAPTER SUMMARY

Swagger (via springdoc-openapi) gives FoodBridge a live, browsable API surface without any hand-written documentation. A `SwaggerConfig` class registers an `OpenAPI` bean describing the project, and a `Bearer Authentication` security scheme tells Swagger UI how to attach a JWT to every request.

*Once Swagger's own endpoints are permitted in `SecurityConfig`, developers can open the UI, click Authorize, paste a token, and exercise every protected endpoint straight from the browser — exactly as Spring Security would see it in production.*